

Templates for Writing Concise Functional Requirements

1. SCOPE

- 1.1. Scope. This document provides recommended syntax and examples for writing concise functional requirements. These templates cover most conditions systems engineers will need to account for in creating systems specifications.

2. APPLICABLE DOCUMENTS

- 2.1. Applicable documents. The formats prescribed in this document conform to the Easy Approach to Requirements Syntax (EARS). For more information on EARS, we recommend the following resources:

[*21 Top Engineering Tips for Writing Exceptionally Clear Requirements*](#), QRA Corp, June 2016. (This document also includes tips for setting up an effective specification document and assuring quality and usability.)

Mavin, Alistair, et al, [EARS \(Easy Approach to Requirements Syntax\)](#), IEEE, October 2009.

Mavin, Alistair, [EARS Tutorial](#), Rolls Royce, September 2010.

Terzakis, John, [EARS: The Easy Approach to Requirements Syntax](#), Intel Corp., July 2013.

3. REQUIREMENTS TEMPLATES

- 3.1. Project Unique Identifier. Most standards for requirements specification require that a “project unique identifier” (PUI) be attached to each requirement in the specification for cross-reference in verification documentation and for easy lookup in the requirements database. As formats and selection methods for PUIs differ from organization to organization, we will not attempt to provide a template for PUIs. We simply include a placeholder – **[PUI]** – in each of the requirements templates which follow, as a reminder to tag each requirement you write with a PUI appropriate for your project.
- 3.2. Ubiquitous requirements. Ubiquitous requirements are those that state a fundamental property of the system. They are universal, meaning they exist at all times and do not require a stimulus to execute.

- 3.2.1. Syntax for ubiquitous requirements.

The <name of system or unit> shall <property of system or unit>. [PUI]

- 3.2.2. Examples of ubiquitous requirements.

The FCC shall control communication on the Avionics Bus in accordance with MIL-STD-1553B and Table 3.1 of the program ICD. [PUI]

The software shall be written in C++. [PUI]

- 3.2.3. Warning concerning ubiquitous requirements. Many requirements may seem ubiquitous but are really driven by some trigger or condition. Requirements written in the ubiquitous format should be examined for conditions or triggers that indicate that the requirement is not truly ubiquitous (universal). When such a requirement is found, re-writing it another format usually makes the trigger-response nature of the requirement more obvious.

Templates for Writing Concise Functional Requirements

- 3.3. Event-driven requirements. Event-driven requirements are those which are invoked or initiated only when a trigger event takes place. They generally state what the system shall do when a specific event occurs and begin with the word “when.”

3.3.1. Syntax for event-driven requirements.

WHEN <trigger> <optional precondition> the <system or unit name> shall <system or unit response>. [PUI]

3.3.2. Examples of event-driven requirements.

***When** the power button is depressed while the system is off, the system shall initiate its start-up sequence. [PUI]*

***When** the water level falls below the Low_Water_Threshold, the software shall open the water valve to fill the tank to the High_Water_Threshold. [PUI]*

- 3.4. Unwanted behaviour requirements. Unwanted behaviour requirements handle error conditions, failures, faults, disturbances and other undesired events. They usually stated in IF/THEN form.

3.4.1. Syntax for unwanted behaviour requirements.

IF <unwanted condition or event>, THEN the <system or unit name> shall <system or unit response>. [PUI]

3.4.2. Examples of unwanted behaviour requirements.

***If** the battery charge level falls below 20% remaining, **then** the system shall go into Power Saver mode. [PUI]*

***If** the input checksum is invalid, **then** the system shall reject the new data and retain the previous data in memory. [PUI]*

- 3.5. State-driven requirements. State-driven requirements are triggered when the system or unit enters a specific state or mode. These requirements are indicated by the word “while” in the requirements statement.

3.5.1. Syntax for state-driven requirements.

WHILE <state>, the <system name> shall <system response>. [PUI]

3.5.2. Examples of state-driven requirements.

***While** in the Power Saver mode, the system shall limit screen brightness to a maximum of 60%. [PUI]*

***While** the autopilot is engaged, the flight control panel shall display a visual indication that the aircraft is under autopilot control. [PUI]*

- 3.6. Optional feature requirements. As the name implies, optional feature requirements specify behavior which involves an optional feature which the system being specified may or may not have installed. Such requirements are invoked only in instances of the system or unit which have the optional feature installed. The word “where” is used to indicate an optional feature requirement.

3.6.1. Syntax for optional feature requirements.

Templates for Writing Concise Functional Requirements

WHERE <feature is included>, the <system or unit name> shall <system or unit response>. [PUI]

3.6.2. Examples of optional feature requirements.

***Where** the automobile is furnished with the GPS navigation system, the automobile shall enable the driver to mute the navigation instructions via the steering wheel controls. [PUI]*

***Where** the encryption hardware is installed, the system shall encrypt data using that encryption hardware, instead of using a software algorithm. [PUI]*

- 3.7. Complex requirements. In defining complex systems, especially real-time systems, engineers will often have to specify complex conditional events involving multiple triggers, states and/or optional features. In such cases, you will often need to use amalgams of the formats described earlier, employing a combination of the keywords (when, if/then, while, where) used in those formats.

3.7.1. Syntax for complex requirements.

<Multiple Conditions>, the <system or unit name> shall <system or unit response>. [PUI]

3.7.2. Examples of complex requirements.

***While** in A/A mode with an A/A missile selected and the Master Arm switch in ARM, **when** the WEAPON RELEASE signal is received from the SSC, the SMS shall send LAUNCH signal to the selected station. [PUI]*

***While** in A/A mode, **if** the selected station returns the WEAPON FAILED signal, **then** the SMS will set the FAIL signal for the selected station, and select the next station in the station selection sequence. [PUI]*

Templates for Writing Concise Functional Requirements

4. SUMMARY

4.1. Summary. Table 1 summarizes the individual requirements templates described in paragraphs 3.2 through 3.7, above.

Table 1: Concise syntax patterns for common types of requirements

Requirement Type	Syntax Pattern
Ubiquitous	The <system name> shall <system response>
Event-Driven	WHEN <trigger> <optional precondition> the <system name> shall <system response>
Unwanted Behaviour	IF <unwanted condition or event>, THEN the <system name> shall <system response>
State-Driven	WHILE <system state>, the <system name> shall <system response>
Optional Feature	WHERE <feature is included>, the <system name> shall <system response>
Complex	<Multiple Conditions>, the <system or unit name> shall <system or unit response>. (combinations of the above patterns)

Suggestions welcome. Please send any suggestions or comments you may have on how we might improve this template to: trevor@qracorp.com.